

Nexus

Diseño e Implementación de un Lenguaje

Informe Final — Stage III

Autómatas, Teoría de Lenguajes y Compiladores
Ingeniería Informática — ITBA

Nombre	Apellido	Legajo	E-mail
Pablo	Gorostiaga	65148	pgorostiaga@itba.edu.ar
Franco	Pampuri	65552	fpampuri@itba.edu.ar
Jonas	Glaubart	65790	jglaubart@itba.edu.ar
Eugenio	Migliaro	65508	emigliaro@itba.edu.ar

Índice

1. Introducción	2
2. Modelo Computacional	2
2.1. Dominio	2
2.2. Lenguaje	2
2.2.1. Gramática	2
2.2.2. Sintaxis general	3
2.2.3. Construcciones	3
2.2.4. Semántica de los atributos de nodo	4
2.2.5. Reglas estructurales	4
3. Implementación	4
3.1. Frontend	4
3.2. Backend	5
3.2.1. Tabla de símbolos	5
3.2.2. Sistema de tipos: <i>graph-property checking</i>	5
3.2.3. Generación de código	7
3.2.4. Runtime	7
3.2.5. Pruebas	8
3.3. Adicionales	8
3.4. Dificultades Encontradas	9
4. Futuras Extensiones	9
5. Conclusiones	9
6. Referencias	10
7. Bibliografía	10

1. Introducción

Nexus es un compilador para un DSL (*Domain-Specific Language*) orientado a grafos, desarrollado para la materia Autómatas, Teoría de Lenguajes y Compiladores. El proyecto se entregó en tres etapas que se reflejan en este informe:

1. **Stage I — Diseño:** concepción del dominio, la gramática y la sintaxis del lenguaje.
2. **Stage II — Frontend:** análisis léxico (Flex), análisis sintáctico (Bison) y construcción del AST.
3. **Stage III — Backend:** análisis semántico, generación de código y la documentación final.

El objetivo de Stage III fue completar el *pipeline* del compilador: validar el programa de entrada contra las reglas del dominio (análisis semántico), emitir un artefacto ejecutable (generación de código) y documentar las decisiones de diseño e implementación tomadas a lo largo de todo el proyecto.

El compilador acepta programas Nexus que declaran grafos con nodos, aristas, grupos y restricciones; definen derivaciones (transformaciones de grafos); y especifican análisis (algoritmos de grafos con exportación de resultados). La salida es un programa Python que, junto con la biblioteca de *runtime* `nexus_runtime.py`, construye los grafos, valida las restricciones, ejecuta los algoritmos y exporta los resultados a formato DOT o JSON.

2. Modelo Computacional

2.1. Dominio

El dominio de Nexus es el **modelado, validación, transformación y análisis de grafos**. El lenguaje describe grafos como objetos matemáticos de manera declarativa, indicando su estructura, sus propiedades y los análisis que se desean ejecutar sobre ellos. En concreto, Nexus permite:

- **Declarar grafos** dirigidos o no dirigidos, con *traits* opcionales (`weighted`, `capacitated`).
- **Definir estructura:** nodos con atributos semánticos opcionales (`root`, `source`, `sink`, `terminal`), aristas con peso y/o capacidad, y grupos de nodos.
- **Especificar restricciones** sobre la estructura del grafo: conectividad, aciclicidad, alcanzabilidad, estructura de árbol y predicados cuantitativos sobre grado.
- **Derivar nuevos grafos** mediante transformaciones: transposición, subgrafo inducido, eliminación de *self-loops* y grafo subyacente.
- **Ejecutar algoritmos:** camino mínimo, orden topológico, componentes conexas, componentes fuertemente conexas, árbol de expansión mínima y flujo máximo.
- **Exportar** grafos y resultados a DOT (para visualización con Graphviz) o JSON.

El foco del diseño estuvo puesto en un núcleo reducido pero expresivo, con una especificación clara, consistente y formalmente verificable: cada construcción del lenguaje tiene precondiciones de dominio que el compilador puede comprobar estáticamente.

2.2. Lenguaje

2.2.1. Gramática

El lenguaje queda definido por una gramática $G = \langle \Sigma, N, \Pi, S \rangle$ de tipo 2 (libre de contexto). El analizador léxico (Flex) define el alfabeto Σ a partir de los lexemas del lenguaje, mientras que el analizador sintáctico (Bison) define el conjunto de no-terminales N , las producciones Π

y el símbolo inicial S . Ambas fases en conjunto conforman la gramática que genera el lenguaje. La gramática es LALR(1) y libre de conflictos.

El símbolo inicial S deriva un **Program**: una o más declaraciones de nivel superior, cada una de las cuales es una declaración de grafo (**graph**), una derivación (**derive**) o un bloque de análisis (**analysis**).

2.2.2. Sintaxis general

```
graph Build:
  kind directed
  traits weighted
  nodes:
    lexer
    parser
    ast
    emit
  edges:
    lexer -> parser weight 2
    parser -> ast weight 3
    ast -> emit weight 4
  groups:
    core = {lexer, parser, ast}
    sinks = {emit}
  constraints:
    assert acyclic
    assert reachable lexer -> emit
    assert forall n in sinks: outdegree(n) = 0

derive CoreBuild from Build using induced_subgraph(core)

analysis Main on Build:
  run shortest_path from lexer to emit as critical_path
  export graph to dot
  export result critical_path to json
```

Los bloques **traits**, **groups** y **constraints** son opcionales; cuando aparecen deben respetar exactamente la sintaxis del lenguaje y ubicarse dentro de la declaración del grafo correspondiente.

2.2.3. Construcciones

Construcción	Significado
graph <id>:	Declara un grafo con identificador único.
kind directed undirected	Tipo del grafo (obligatorio).
traits weighted capacitated	Propiedades opcionales del grafo.
nodes :	Sección de declaración de nodos.
<id> [root source sink terminal]	Nodo con atributo semántico opcional.
edges :	Sección de declaración de aristas.
<u> -> <v> [weight N] [capacity N]	Arista dirigida (peso/capacidad opcional).
<u> - <v> [weight N] [capacity N]	Arista no dirigida (peso/capacidad opcional).
groups :	Sección de declaración de grupos.
<id> = {<n1>, <n2>, ...}	Grupo nombrado de nodos.
constraints :	Sección de restricciones.
assert connected	Grafo conexo (solo undirected).

Construcción	Significado
<code>assert strongly_connected</code>	Fuertemente conexo (solo <code>directed</code>).
<code>assert acyclic</code>	Grafo acíclico.
<code>assert reachable <u> -> <v></code>	Existe un camino de u a v .
<code>assert tree rooted_at <r></code>	Es árbol con raíz r (solo <code>directed</code>).
<code>assert binary_tree rooted_at <r></code>	Es árbol binario con raíz r (solo <code>directed</code>).
<code>assert forall <v> in <g>: <pred></code>	Predicado de grado para todo nodo del grupo.
<code>derive <id> from <src> using <t></code>	Crea un grafo derivado.
<code>analysis <id> on <graph>:</code>	Bloque de análisis sobre un grafo.
<code>run <algo> [from <u> to <v>] as <r></code>	Ejecuta un algoritmo y nombra su resultado.
<code>export graph to dot json</code>	Exporta el grafo.
<code>export result <id> to dot json</code>	Exporta un resultado.

Predicados de grado: `degree(n)`, `indegree(n)`, `outdegree(n)` comparados con `=`, `!=`, `>=`, `<=`, `<`, `>` y un valor entero.

Transformaciones: `transpose`, `induced_subgraph(<group>)`, `remove_self_loops`, `underlying`.

Algoritmos: `shortest_path`, `topological_sort`, `components`, `scc`, `mst`, `max_flow`.

2.2.4. Semántica de los atributos de nodo

Los atributos de nodo no son meras etiquetas descriptivas, sino anotaciones con semántica verificable por el compilador:

- **root:** vértice raíz del grafo. A lo sumo uno por grafo; en grafos dirigidos debe cumplir `indegree(root) = 0`. Toda restricción de `tree/binary_tree` debe enraizarse en este nodo.
- **source:** origen principal de un grafo dirigido. A lo sumo uno; debe cumplir `indegree = 0` y `outdegree > 0`.
- **sink:** destino principal de un grafo dirigido. A lo sumo uno; debe cumplir `outdegree = 0` e `indegree > 0`.
- **terminal:** nodo hoja. En grafos dirigidos `outdegree = 0`; en no dirigidos `degree = 1`. Puede haber varios.

2.2.5. Reglas estructurales

Se permiten *self-loops* pero no aristas paralelas: entre dos vértices puede existir a lo sumo una arista (pares ordenados en grafos dirigidos, pares no ordenados en no dirigidos). Si un grafo declara `assert acyclic`, el compilador rechaza todo *self-loop* y todo ciclo, ya que constituyen violaciones de la aciclicidad.

3. Implementación

El `EntryPoint` ejecuta el *pipeline* en tres fases: **parse** → **análisis semántico** → **generación de código**. La fase semántica sólo habilita la generación si el programa es válido respecto del dominio.

3.1. Frontend

El frontend fue construido en Stage II y no requirió modificaciones de fondo en Stage III.

Análisis léxico (Flex). El *lexer* (`FlexPatterns.1`) reconoce las palabras reservadas del lenguaje (`graph`, `kind`, `directed`, `undirected`, `traits`, `nodes`, `edges`, `groups`, `constraints`,

`assert`, los nombres de constraints, transformaciones y algoritmos, los atributos de nodo, etc.), los operadores (`->`, `-`, `=`, `!=`, `>=`, `<=`, `<`, `>`), los delimitadores (`:`, `{`, `}`, `,`, `(`, `)`), identificadores (`[a-zA-Z_][a-zA-Z0-9_]*`), enteros y comentarios de línea (`//`).

Análisis sintáctico (Bison). La gramática (`BisonGrammar.y`) define las producciones del lenguaje y las acciones semánticas que construyen el AST. Es LALR(1) y libre de conflictos.

AST (`AbstractSyntaxTree.h/c`). Define los tipos del árbol con un diseño de unión etiquetada (*tagged union*). El nodo raíz es `Program { TopLevelDeclList * decls }`. Cada `TopLevelDecl` es una unión sobre `GraphDecl`, `DeriveDecl` y `AnalysisDecl`. Se proveen constructores, destructores y cobertura de `%destructor` en Bison para evitar *memory leaks*. El AST es el contrato de entrada del backend.

3.2. Backend

3.2.1. Tabla de símbolos

La tabla de símbolos se implementa mediante `IdSet`, un *hash-set* de *strings* con resolución de colisiones por encadenamiento (`IdSet.h/c`). Ofrece tres operaciones:

- `idSetAdd(set, id)` — inserta una copia del identificador; retorna `false` si ya existía (detección de duplicados).
- `idSetContains(set, id)` — consulta de pertenencia.
- `idSetForEach(set, visit, ctx)` — itera sobre todos los identificadores (usado para propagar nodos/grupos a grafos derivados).

El analizador semántico (`SemanticAnalyzer.c`) mantiene las siguientes tablas, organizadas como una pila de *scopes*:

Tabla	Scope	Contenido
<code>graphIds</code>	Global	Grafos declarados y derivados
<code>analysisIds</code>	Global	Identificadores de análisis
<code>GraphInfo.nodes</code>	Por grafo	Nodos declarados
<code>GraphInfo.groups</code>	Por grafo	Grupos declarados
<code>results</code>	Por análisis	Resultados de <code>run ... as <id></code>

Adicionalmente, `GraphInfo` almacena el `kind` y los `traits` de cada grafo en un registro enlazado, permitiendo validar operaciones *cross-graph* (derivaciones y análisis que referencian grafos por nombre, incluyendo grafos derivados).

3.2.2. Sistema de tipos: *graph-property checking*

En Nexus no existe un sistema de tipos tradicional con coerción o inferencia. El “sistema de tipos” se reinterpreta como **validación de propiedades de grafos**: cada operación impone precondiciones sobre el `kind` y los `traits` del grafo objetivo. Se implementaron **48 validaciones semánticas**, cada una con un test de rechazo dedicado. La siguiente matriz las resume agrupadas por categoría y por la función del analizador que las aplica.

Identificadores duplicados.

Validación	Función
Grafo duplicado	<code>_checkTopLevel</code>
Análisis duplicado	<code>_checkTopLevel</code>
Nodo duplicado	<code>_checkGraphScope</code>
Grupo duplicado	<code>_checkGraphScope</code>
Resultado duplicado	<code>_checkAnalysisScope</code>
Arista paralela duplicada	<code>_checkGraphScope</code>

Referencias no declaradas.

Validación	Función
Endpoint de arista	<code>_checkGraphScope</code>
Miembro de grupo	<code>_checkGraphScope</code>
Grafo fuente de <code>derive</code>	<code>_checkTopLevel</code>
Grafo de <code>analysis</code>	<code>_checkAnalysisScope</code>
Nodo de <code>constraint</code>	<code>_checkConstraints</code>
Grupo de <code>forall</code>	<code>_checkConstraints</code>
Endpoint de <code>run</code>	<code>_checkAnalysisScope</code>
Nodo fuera del subgrafo <code>induced_subgraph</code>	<code>_checkAnalysisScope</code>
Resultado de <code>export</code>	<code>_checkAnalysisScope</code>

Traits, operadores y atributos de nodo.

Validación	Función
<code>weight</code> sin trait <code>weighted</code>	<code>_checkGraphScope</code>
<code>capacity</code> sin trait <code>capacitated</code>	<code>_checkGraphScope</code>
<code>-></code> en grafo <code>undirected</code>	<code>_checkGraphScope</code>
<code>-</code> en grafo <code>directed</code>	<code>_checkGraphScope</code>
<code>source/sink</code> en <code>undirected</code>	<code>_checkGraphScope</code>
Múltiples <code>root/source/sink</code>	<code>_checkGraphScope</code>
<code>root</code> con <code>indegree $\neq$ 0</code>	<code>_checkGraphScope</code>
<code>source: indegree $\neq$ 0 u outdegree = 0</code>	<code>_checkGraphScope</code>
<code>sink: outdegree $\neq$ 0 o indegree = 0</code>	<code>_checkGraphScope</code>
<code>terminal</code> con grado inválido	<code>_checkGraphScope</code>

Constraints, transformaciones y algoritmos vs. kind/traits.

Validación	Función
connected en directed	<code>_checkConstraints</code>
strongly_connected en undirected	<code>_checkConstraints</code>
tree/binary_tree en undirected	<code>_checkConstraints</code>
rooted_at distinto del nodo root	<code>_checkConstraints</code>
acyclic con ciclo o <i>self-loop</i>	<code>_checkConstraints</code>
transpose/underlying en undirected	<code>_checkTopLevel</code>
induced_subgraph con grupo no declarado	<code>_checkTopLevel</code>
topological_sort/scc en undirected	<code>_checkAlgorithm</code>
topological_sort en grafo cíclico	<code>_checkAlgorithm</code>
components en directed	<code>_checkAlgorithm</code>
mst en directed o sin weighted	<code>_checkAlgorithm</code>
max_flow en undirected o sin capacitated	<code>_checkAlgorithm</code>
max_flow con endpoints $\neq$ source/sink	<code>_checkAlgorithm</code>
shortest_path sin weighted	<code>_checkAlgorithm</code>

Estructurales. Grafo sin nodos (`_checkGraphScope`) y análisis sin sentencias (`_checkAnalysisScope`): aunque la gramática los admite, no son constructos semánticamente útiles y se rechazan.

3.2.3. Generación de código

El generador (`Generator.c`) realiza un recorrido del AST dirigido por sintaxis (*syntax-directed*), emitiendo Python a `stdout`. La función `executeGenerator` recorre la lista de `TopLevelDecl` y despacha a:

- `_generateGraph` — emite la construcción del grafo (`Graph(directed=..., weighted=..., capacitated=...)`), seguida de `add_node`, `add_edge`, `add_group` y las *assertions* de las restricciones.
- `_generateDerive` — emite una asignación con la llamada a la transformación correspondiente.
- `_generateAnalysis` — emite los `run` como asignaciones al resultado del algoritmo y los `export` como llamadas a `write_dot/write_json`.

El encabezado generado importa los símbolos del *runtime*:

```
from nexus_runtime import Graph
from nexus_runtime import transpose, induced_subgraph, remove_self_loops, underlying
from nexus_runtime import shortest_path, topological_sort, components, scc, mst,
    max_flow
from nexus_runtime import assert_connected, assert_strongly_connected, assert_acyclic
from nexus_runtime import assert_reachable, assert_tree, assert_binary_tree,
    assert_forall
from nexus_runtime import write_dot, write_json
```

Decisión de diseño. Se optó por emitir un programa Python plano (sin clases, funciones ni indentación) porque el modelo de ejecución de Nexus es estrictamente secuencial: declarar, derivar, analizar. Esto simplifica el generador y hace la salida directamente legible.

3.2.4. Runtime

La pregunta de diseño que plantea la consigna — la relevancia o no de un *runtime* para el dominio — se respondió afirmativamente: el dominio de grafos requiere estructuras de datos y

algoritmos no triviales que conviene delegar a una biblioteca, en lugar de inlinearlos en cada programa generado. El *runtime* (`nexus_runtime.py`) es una biblioteca Python de *solo biblioteca estándar* que se distribuye como *asset* junto al compilador; no se enlaza al compilador, es una dependencia del programa generado.

Componente	Función
Graph	<code>add_node</code> , <code>add_edge</code> , <code>add_group</code> , <code>neighbors</code> . Representación: lista de adyacencia con diccionarios <code>{to, weight, capacity}</code> .
Derivaciones	<code>transpose</code> , <code>induced_subgraph</code> , <code>remove_self_loops</code> , <code>underlying</code> .
Algoritmos	<code>shortest_path</code> (Dijkstra), <code>topological_sort</code> (Kahn), <code>components</code> (BFS), <code>scc</code> (Tarjan), <code>mst</code> (Kruskal + Union-Find), <code>max_flow</code> (Edmonds-Karp).
Assertions	<code>assert_connected</code> , <code>assert_strongly_connected</code> , <code>assert_acyclic</code> , <code>assert_reachable</code> , <code>assert_tree</code> , <code>assert_binary_tree</code> , <code>assert_forall</code> .
Exportadores	<code>write_dot</code> , <code>write_result_dot</code> , <code>write_json</code> .

Validaciones *compile-time* vs. *runtime*. Las restricciones del lenguaje se verifican en dos momentos complementarios:

- *Compile-time*: el analizador semántico verifica que la restricción sea *aplicable* al tipo de grafo (p.ej. `tree` sólo en `directed`). Previene errores de tipo.
- *Runtime*: las *assertions* ejecutables verifican que el grafo concreto *cumple* la restricción (p.ej. que efectivamente es un árbol). Previene errores de valor que no pueden conocerse estáticamente.

3.2.5. Pruebas

La suite de pruebas final comprende:

Suite	Programas
Aceptación (frontend + backend)	35
Rechazo léxico/sintáctico	12
Rechazo semántico	48
Generación de código <i>end-to-end</i>	24

El `harness test.sh` ejecuta las suites de aceptación y rechazo (verificando códigos de salida) y `test-codegen.sh` genera Python desde los programas de aceptación, lo ejecuta y comprueba que se produzcan los artefactos DOT/JSON esperados. Una *pipeline* de Integración Continua (`.github/workflows/pipeline.yaml`) instala el toolchain, construye y ejecuta ambos *harness* en cada *push* y *pull request*.

3.3. Adicionales

- **Exportador DOT:** genera archivos compatibles con Graphviz, con peso y capacidad como *labels* de arista y atributos de nodo.
- **Exportador JSON:** serializa grafos como `{nodes: [...], edges: [...]}` y resultados de algoritmos como estructuras Python.
- **Grafos derivados con validación completa:** al procesar un `derive`, el analizador sintetiza un `GraphInfo` para el grafo derivado, copiando nodos y grupos del grafo fuente (vía `idSetForEach`); `underlying` además cambia el `kind` a `undirected`. Esto permite que

análisis posteriores referencien grafos derivados con la misma validación que los grafos declarados.

- **Metodología TDD:** cada validación semántica se desarrolló escribiendo primero el test de rechazo (RED), implementando la regla y verificando el paso a verde (GREEN).

3.4. Dificultades Encontradas

1. **Resolución de nodos *cross-graph*.** Los grafos derivados necesitan conocer los nodos del grafo fuente para validar análisis posteriores. Se resolvió con `idSetForEach` para copiar los conjuntos de nodos y grupos al `GraphInfo` del derivado.
2. **Logs contaminando *stdout*.** El sistema de *logging* del proyecto base envía DEBUG/INFO a *stdout*, donde el generador también escribe el código Python. Se resolvió capturando la salida con `LOGGING_LEVEL=CRITICAL`.
3. **Predicados de grado a *runtime*.** Predicados como `forall n in g: degree(n) >= K` no pueden verificarse en *compile-time* porque requieren la estructura concreta del grafo; se delegaron al *runtime* como *assertions* parametrizadas.
4. **Ownership de memoria bajo Bison.** El AST mantiene el *ownership* de todos los *strings* (duplicados con `strdup` en las acciones semánticas). El analizador y el generador usan punteros prestados; los `IdSet` duplican sus claves para ser independientes del AST.
5. **Decisiones gramática vs. semántica.** Algunas restricciones podrían haberse forzado en la gramática, pero se optó por una gramática permisiva con validación semántica posterior: simplifica la gramática y produce mejores mensajes de error.

4. Futuras Extensiones

- **Coerción de tipos:** permitir `shortest_path` en grafos no `weighted` asumiendo peso unitario.
- **Más algoritmos:** recorridos BFS/DFS, detección de ciclos con reporte del ciclo, flujo de costo mínimo.
- **Salida multi-archivo:** emitir cada análisis a un archivo separado en vez de un único programa.
- **Lenguaje de *constraints* más rico:** combinaciones lógicas (`and`, `or`, `not`) y predicados sobre aristas.
- **Targets alternativos:** además de Python, emitir C++, Rust o directamente DOT/JSON sin *runtime*.
- **Importación de grafos:** cargar grafos desde archivos DOT o JSON.
- **Subsistema generador/build:** construcción paramétrica de grafos con variables locales, condicionales y estructuras iterativas acotadas (propuesto ya en Stage I como extensión).

5. Conclusiones

El compilador Nexus implementa exitosamente las tres etapas del proyecto: un frontend libre de conflictos que construye un AST completo, un analizador semántico con 48 validaciones que cubren identificadores, referencias, *traits*, operadores, atributos de nodo, restricciones, transformaciones, algoritmos y reglas estructurales, y un generador que produce programas Python ejecutables.

La decisión de generar Python con un *runtime* de solo biblioteca estándar resultó acertada: los programas generados se ejecutan sin dependencias externas (salvo Python 3 y, opcionalmente, Graphviz para visualizar los DOT), lo que mantiene al compilador libre de dependencias enlazadas y facilita el *testing end-to-end*.

El enfoque TDD fue particularmente efectivo para las validaciones semánticas: cada regla quedó documentada por un test de rechazo dedicado que describe exactamente qué programa inválido detecta. La separación entre validaciones de *compile-time* (aplicabilidad) y *runtime* (cumplimiento) permitió modelar fielmente el dominio sin sobrecargar ninguna de las dos fases.

6. Referencias

- Stage I — Diseño del lenguaje Nexus: `doc/TLA_Stage_I.pdf` (documento de diseño original).
- Flex-Bison-Compiler v2.0.0: <https://github.com/agustin-golmar/Flex-Bison-Compiler/tree/v2.0.0> (proyecto base).
- Material de la cátedra: apuntes de Análisis Semántico (v0.2.0), Tabla de Símbolos, Sistema de Tipos, Generación de Código (v0.1.0) y Runtime (v0.1.0).

7. Bibliografía

- Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2006). *Compilers: Principles, Techniques, and Tools* (2.^a ed.). Addison-Wesley.
- Levine, J. (2009). *flex & bison*. O'Reilly Media.
- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3.^a ed.). MIT Press.